# Code-Layout, Readability and Reusability

| | |
|---|---|
| **Date** | 05 July 2026 |
| **Team ID** | SWTID-2026-2423 |
| **Project Name** | Emotion Detection & Learning Support Engine |
| **Maximum Marks** | 5 Marks |

## Code-Layout, Readability and Reusability:

This document evaluates the quality of the codebase in terms of its structure, readability, and reusability. Well-organized code with proper naming conventions, comments, and modular design ensures that the project is maintainable and can be extended or reused in future development.

## Code Layout Checklist:

| S.No | Code Quality Parameter | Description | Followed (Yes / No / Partial) | Remarks |
|---|---|---|---|---|
| 1 | Consistent Indentation | Uniform spacing/tabs used throughout the code | yes | Standard PEP 8 formatting applied across all Python modules. |
| 2 | Proper File Structure | Files and folders are logically organized | yes | Clean segregation into separate frontend, backend, and utility folders. |
| 3 | Meaningful Variable Names | Variables reflect their purpose clearly | yes | Avoided generic names; used explicit terms like user_badge_id or issuer_signature. |
| 4 | Function / Method Names | Functions are descriptively named | yes | Used active action names like verify_credential_hash() or issue_digital_badge(). |
| 5 | Code Comments | Inline and block comments explain logic | yes | Docstrings are present for all major backend route endpoints and cryptographic tasks. |
| 6 | Modular Design | Code is split into reusable functions/modules | yes | Database interactions and security utilities are written as standalone modules. |
| 7 | No Redundant Code | Duplicate or unused code is removed | yes | Cleaned up boilerplate code; shared processes utilize central helper functions. |
| 8 | Error Handling | Exceptions and errors are handled gra cefully | yes | Broad try-except blocks protect API routes, returning proper HTTP status codes. |

## Reusable Components / Modules:

| S.No | Component / Module Name | Language / Technolog | yWhere Reused | Reusability Level (High / Medium / Low) |
|---|---|---|---|---|
| 1 | Hash Generator Utility | Python (hashlib) | Used by both the Issuer module (to create badges) and the Recruiter module (to verify authenticity). | High |

| | | | | |
|---|---|---|---|---|
| 2 | Database Connection Manager | Python / SQLAlchemy | Shared across all user registration, login, and dashboard data retrieval routines. | High |
| 3 | Auth Token Validator (JWT) | Python / Flask-JWT | Attached as a middleware decorator to secure admin, student, and corporate backend APIs. | High |
| 4 | Alert Notification Toast | JavaScript / Tailwind CSS | Reused throughout the UI to display success/error popups during actions. | Medium |
| 5 | PDF Data Extractor | Python (PyPDF2) | Used during profile building to parse certificates and automatically pre-fill user forms. | Medium |

### Overall Code Quality Assessment:

| Aspect | Rating (1-5) | Comments |
|---|---|---|
| Code Layout & Structure | 5 / 5 | Highly organized folder directory system separating operational concerns cleanly. |
| Readability | 4.5 / 5 | Self-explanatory code structure due to precise PEP 8 variable naming standards. |
| Reusability | 5 / 5 | Centralized hashing and authentication systems drastically reduce codebase footprint. |
| Documentation / Comments | 4.5 / 5 | Clear docstrings outline parameter expectations for all major API routes. |
| Overall Score | 4.5 / 5 | Excellent, production-ready implementation that meets clean code standards. |